Programmation procédurale (UAA11)

Atelier : les bases

Objectifs de l'atelier :

- ✔ Afficher du texte et des valeurs avec printf.
- ✔ Choisir le bon type et le bon code de formatage.
- ✔ Lire une valeur au clavier avec scanf.
- ✔ Découvrir la division entière, le modulo et les conversions.
- ✔ Comprendre qu'un caractère est un nombre et pourquoi on écrit getchar() après un scanf.

Avant de commencer

Cet atelier se fait seul, avant la séance de théorie. Tu n'as rien à apprendre par cœur : tu vas compiler de petits programmes, les exécuter, les modifier, et découvrir par toi-même comment le C se comporte. Les règles que tu formuleras serviront de point de départ à la séance.

Les programmes à télécharger :

https://lucasembrechts.github.io/UAA11/fichiers/atelier_bases.zip

Comment travailler :

- ➢ décompresse l'archive : tu obtiens un fichier `.c` par programme. Le nom du fichier est indiqué au-dessus de chaque programme ;
- ➢ ouvre chaque fichier dans ton éditeur habituel, compile-le et exécute-le ;
- ➢ **avant** chaque exécution, écris ta prédiction. Ne la corrige pas après coup : une prédiction fausse est justement ce qui va t'apprendre quelque chose ;
- ➢ certains programmes ne compilent pas : c'est voulu. Lis alors attentivement le message du compilateur, il fait partie de la découverte ;
- ➢ ne cherche pas la réponse dans le cours ou sur Internet : l'atelier est conçu pour que tu la trouves en observant ;
- ➢ à la fin de chaque section, résume ta découverte dans le cadre « Ma règle ». L'ensemble de tes règles formera ta fiche pour la séance théorique.

Section 0 : afficher du texte

Fichier : section0.c

```
#include <stdio.h>

void main(void) {
   printf("Bonjour");
   printf("tout le monde");
   printf("\n");
   printf("Il a dit \"salut\" et 50 %% de la classe a ri.\n");
}
```

Prédis

Qu'affiche ce programme ? Combien de lignes ?

Exécute, puis explique

Pourquoi « Bonjour » et « tout le monde » sont-ils collés sur la même ligne ?

Dans la dernière ligne du programme, qu'affichent `\"` et `%%` ? Pourquoi ne peut-on pas écrire simplement `"` et `%` ?

Modifie

Retire le `\` devant le premier guillemet de `salut`, puis compile. Que se passe-t-il ?

✎ Ma règle : comment afficher du texte, un retour à la ligne, un guillemet et un pourcent ?

Section 1 : variables et affichage

Fichier : section1.c

```
#include <stdio.h>
#include <stdbool.h>

void main(void) {
   int age = 17;
   float taille = 1.75;
   char initiale = 'L';
   bool inscrit = true;
   printf("Age : %d ans\n", age);
   printf("Taille : %f m\n", taille);
   printf("Taille : %.2f m\n", taille);
   printf("Initiale : %c\n", initiale);
   printf("Inscrit : %d\n", inscrit);
}
```

Prédis

Qu'affiche ce programme ? Attention aux deux lignes « Taille ».

Exécute, puis explique

Quelle est la différence entre `%f` et `%.2f` ?

> 📖 Les `%d`, `%f`, `%c`… placés dans le texte d'un `printf` s'appellent des **codes de formatage** : chacun indique à `printf` comment afficher la valeur correspondante.

Pourquoi `inscrit` s'affiche-t-il `1` et pas `true` ?

Associe chaque type à son code de formatage : `int`, `float`, `char`.

Modifie

Remplace `%d` par `%f` dans la ligne de l'âge. Le programme compile-t-il ? Qu'affiche-t-il ?

Fichier : section1_suite.c

```
#include <stdio.h>

void main(void) {
   int nbPlaces = 9.99;
   printf("Places : %d\n", nbPlaces);
   printf("|%6.2f|%-6d|%04d|\n", 3.14159, 42, 7);
}
```

Prédis l'affichage de ce second programme, puis vérifie. Que devient `9.99` dans un `int` ? Que font `6.2`, `-6` et `04` ?

✎ Ma règle : comment choisir le code de formatage d'une variable ?

Section 2 : lire au clavier

Fichier : section2.c

```
#include <stdio.h>

void main(void) {
   int annee;
   printf("Ton annee de naissance : ");
   scanf("%d", &annee);
   getchar();
   printf("En 2026, tu auras %d ans.\n", 2026 - annee);
}
```

Prédis, puis exécute

Tu vas taper `2009`. Qu'affichera le programme ?

Explique

Dans `scanf("%d", &annee)`, à quoi sert `%d` ? Et le `&` ?

Modifie

Retire le `&` devant `annee`. Compile (lis les avertissements) et exécute. Que se passe-t-il ?

Remets le `&`, puis tape `dix-sept` au lieu d'un nombre. Qu'observes-tu ?

✎ Ma règle : comment lire une valeur au clavier ?

Section 3 : diviser des entiers

Fichier : section3.c

```
#include <stdio.h>

void main(void) {
   int bonbons = 17;
   int enfants = 5;
   printf("%d\n", bonbons / enfants);
   printf("%d\n", bonbons % enfants);
   float part = bonbons / enfants;
   printf("%.2f\n", part);
   part = (float) bonbons / enfants;
   printf("%.2f\n", part);
}
```

Prédis

17 bonbons à partager entre 5 enfants. Qu'affiche ce programme ?

Exécute, puis explique

Que calculent `/` et `%` entre deux entiers ?

> 📖 L'opération `%` s'appelle le **modulo** : elle donne le reste de la division entière.

`part` est un `float`. Pourquoi vaut-il 3.00 et pas 3.40 la première fois ?

Que change le `(float)` de la dernière division ?

Modifie

Fichier : section3_heures.c

```
#include <stdio.h>

void main(void) {
   int minutes = 125;
   printf("%d h %02d\n", minutes / 60, minutes % 60);
}
```

Prédis l'affichage, puis vérifie. Comment `/` et `%` transforment-ils des minutes en heures ?

Fichier : section3_zero.c

```
#include <stdio.h>

void main(void) {
   int bonbons = 17;
   int enfants = 0;
   printf("Chacun recoit %d bonbons\n", bonbons / enfants);
   printf("Fin\n");
}
```

Et s'il n'y a aucun enfant ? Exécute et compare avec ton voisin.

✎ Ma règle : comment se comporte la division entre deux entiers ?

Section 4 : ++ et compagnie

Fichier : section4.c

```c
#include <stdio.h>

void main(void) {
   int n = 5;
   int a = n++;
   int b = ++n;
   printf("a = %d, b = %d, n = %d\n", a, b, n);
   n = 10;
   n += 3;
   n *= 2;
   printf("n = %d\n", n);
}
```

Prédis, puis exécute

Qu'affiche ce programme ? Suis la valeur de `n` ligne par ligne.

Explique

`n++` et `++n` augmentent tous les deux `n` de 1. Quelle différence y a-t-il entre `a = n++` et `b = ++n` ?

Que signifient `n += 3` et `n *= 2` ?

✎ Ma règle : que font ++, += et *= ?

Section 5 : un caractère est un nombre

Fichier : section5.c

```
#include <stdio.h>

void main(void) {
    char lettre = 'C';
    printf("%c %d\n", lettre, lettre);
    printf("%c\n", lettre + 1);
    printf("%d\n", lettre + 1);
    printf("%c\n", lettre + 32);
    char chiffre = '7';
    printf("%d\n", chiffre - '0');
}
```

Prédis

Qu'affiche ce programme ?

Exécute, puis explique

Que contient réellement la variable `lettre` ? Que fait `lettre + 1` ?

Pourquoi `lettre + 32` donne-t-il la minuscule ?

Modifie

Ajoute `printf("%d\n", chiffre);`. Qu'affiche-t-il ? Pourquoi faut-il écrire `chiffre - '0'` pour obtenir le nombre 7 ?

✎ Ma règle : qu'est-ce qu'un char en réalité ?

Section 6 : le retour à la ligne fantôme

Fichier : section6.c

```
#include <stdio.h>

void main(void) {
   int quantite;
   char taille;
   printf("Quantite : ");
   scanf("%d", &quantite);
   printf("Taille (S, M ou L) : ");
   scanf("%c", &taille);
   printf("Commande : %d x taille [%c]\n", quantite, taille);
}
```

Prédis, puis exécute

Tape `3`, puis `M`. Que se passe-t-il ? Le programme te laisse-t-il taper la taille ?

Explique

Quand tu as tapé `3`, tu as aussi appuyé sur Entrée. Où est passé ce retour à la ligne ? Qu'a lu le second `scanf` ?

> 📖 La zone où attendent les caractères tapés au clavier mais pas encore lus s'appelle le **tampon** (en anglais : buffer).

Modifie

Ajoute `getchar();` juste après le premier `scanf`. Exécute à nouveau. Que fait `getchar();` ici ?

✎ Ma règle : pourquoi écrit-on getchar(); après chaque scanf ?

Défi final

Crée un fichier `defi.c` : le programme aide un groupe à partager l'addition au restaurant.

- il demande le montant de l'addition (un nombre entier d'euros) et le nombre de personnes ;
- il affiche la part de chacun avec deux décimales ;
- il affiche combien de billets de 20 euros il faut, et ce qu'il reste à payer en plus ;
- il affiche le montant avec 10 % de pourboire ;
- un `getchar();` suit chaque `scanf`.

Exemple d'exécution :

```
Montant de l'addition (euros) : 87
Nombre de personnes : 4
Chacun paie 21.75 euros.
En billets de 20 : 4 billets, il reste 7 euros.
Avec 10 % de pourboire : 95.70 euros.
```

Que se passerait-il si on écrivait `addition / personnes` sans `(float)` pour la part de chacun ?